CUDA platform installed in our system. As before, the installation of PyTorch will be handled by Anaconda Navigator. Let us begin by discussing a simple Python script using PyTorch. Consider the Python script shown in Figure 5.

We have already discussed tensors and how they can be created using TensorFlow. We can also create tensors using PyTorch. Now, let us try to understand the Python script shown in Figure 5, which creates a tensor using PyTorch. Line 1 imports the library PyTorch. Line 2 defines a two-dimensional list named *val*. Line 3 converts the list *val* to a tensor named *ten*. Line 4 prints the content of the tensor *ten*. Figure 5 also shows the output of the script. Notice that the values in the list *val* remain unchanged even after getting converted to tensor *ten*. Finally, Line 5 prints the type of the tensor *ten*, which is of type *torch.Tensor*.

Figure 6 shows a Python script generating a tensor with random values. Line 1 of the script uses the method *rand( )* to generate a 3x3 matrix and store it in a tensor named *random_tensor*. Line 2 prints the content of the tensor r*andom_tensor*. The output of the script is also shown in Figure 6. Notice that the content will change with each iteration because we are using the *rand( )* method to generate random tensors. Finally, Line 3 prints that the type of *random_tensor* is also *torch.Tensor*.

It is also possible to convert a NumPy array to a PyTorch tensor. Consider the Python script shown in Figure 7. Line 1 imports NumPy. Line 2 creates a two-dimensional NumPy array and stores it in a variable named *val*. Line 3 prints the type of the *val*. Figure 7 also shows the output of the script. It can be seen that *val* is of type *numpy.ndarray* (NumPy array). Line 4 uses the method *from_numpy*( ) to convert the array *val* to a PyTorch tensor and stores it in a variable named *ten*. Line 5 prints the type of the variable *ten*. Notice that *ten* is of type *torch.Tensor*. Finally, Line 6 prints the content of the tensor *ten*. The values in the tensor *ten* are the same as the values in the array *val*. We will be revisiting PyTorch in the next article in this series when we discuss natural language processing.

Now, it is time to wind-up this article. In this article, we have developed more intuition about neural networks. Further, we used scikit-learn to learn more about unsupervised learning. We also had our first interaction with PyTorch. In the next article in this series on AI and machine learning, we will continue exploring model training and neural networks. Natural language processing (NLP) is an important area in AI and machine learning. However, so far in this series we haven't discussed anything about AI based NLP. So in the next article, we will discuss NLP using PyTorch as well as familiarise ourselves with NLTK (natural language toolkit), a Python library used exclusively for NLP. END

```
1 import torch
2 val = [[11, 22, 33], [44, 55, 66], [77, 88, 99]]
3 ten = torch.tensor(val)
4 print(ten)
5 print(type(ten))
```

```
tensor([[11, 22, 33],
        [44, 55, 66],
        [77, 88, 99]])
<class 'torch.Tensor'>
```

Figure 5: A first introduction to PyTorch

```
1 random_ten = torch.rand(3, 3)
2 print(random_ten)
3 print(type(random_ten))
```

```
tensor([[0.3473, 0.6767, 0.1339],
        [0.3764, 0.5648, 0.4349],
        [0.4054, 0.2175, 0.2619]])
<class 'torch.Tensor'>
```

Figure 6: A random tensor

```
1 import numpy as np
2 val = np.array([[11, 22], [32, 44]])
3 print(type(val))
4 ten = torch.from_numpy(val)
5 print(type(ten))
6 print(ten)
```

```
<class 'numpy.ndarray'>
<class 'torch.Tensor'>
tensor([[11, 22],
        [32, 44]])
```

Figure 7: Array to tensor conversion

By: Deepu Benson

The author is currently working as assistant professor in the Amal Jyothi College of Engineering, Kanjirappally, Kerala. He is a free software enthusiast and his area of interest is theoretical computer science.